

AUDIT-SID: Artifact-Level Diagnostics for Semantic-ID Tokenizers in Generative Recommendation

Anonymous Author(s)

Abstract

Semantic-ID tokenizers have become an interface layer in generative recommendation: they turn items into discrete addresses that a sequence model can generate. Aggregate Recall@K or NDCG, however, often gives little information about the address space itself. Codebook collapse, full-code collisions, behaviorally weak prefixes, tail compression, and large prefix fan-out can remain hidden until after expensive generator training. We present AUDIT-SID, a mapping-first diagnostic toolkit for item-to-SID tokenizer artifacts. AUDIT-SID normalizes item mappings, metadata, interactions, and optional generator outputs, then reports utilization, collision profile, semantic-collaborative alignment, head-tail allocation, and structural cost proxies. On a same-item Musical case study with 23,742 items, a GRID/RQ-KMeans-style feature-text export has 3,749 unique full SIDs and a 0.9769 full-collision rate, while a bounded ReSID/GAOQ export is collision-free under its exported mapping; the GRID collision rate remains 0.9751–0.9769 across three seeds. Controlled stressors further show that raw collision volume, interaction-qualified collision risk, tail capacity, and active-prefix cost are distinct artifact signals. The contribution is a reusable artifact audit suite, not a new tokenizer or a generative-recommender leaderboard.

CCS Concepts

• **Information systems** → **Recommender systems**; *Retrieval models and ranking*.

Keywords

semantic IDs, generative recommendation, tokenizer diagnostics, recommender systems

ACM Reference Format:

Anonymous Author(s). 2026. AUDIT-SID: Artifact-Level Diagnostics for Semantic-ID Tokenizers in Generative Recommendation. In . ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Semantic-ID (SID) tokenization has become a practical design point for generative recommendation. Systems such as TIGER map items into discrete code sequences so that a sequence model can generate candidate identifiers rather than score every catalog item directly [11]. This shift has made the tokenizer itself a first-order

artifact. Recent work explores residual quantization frameworks, recommender-native encoding, differentiable SID learning, non-uniform quantization, qualified collisions, and continual tokenization [3–5, 7, 9, 14]. The broader identifier literature also shows that item indexing choices matter before generative retrieval is trained end-to-end [6], that semantic IDs can improve industrial ranking generalization [12], and that collaborative tokenization methods explicitly optimize behavioral alignment and assignment diversity [13, 15].

The evaluation practice has not caught up with this artifact boundary. Prior recommender tooling has argued for behavioral tests beyond NDCG-style aggregates [1], but SID tokenizers introduce a distinct artifact: the generated item address space. A tokenizer can support a usable downstream Recall@K while still hiding failure modes that are hard to diagnose from aggregate ranking metrics: underused codebooks, repeated full codes, semantically neat but behaviorally poor prefixes, tail items sharing too little capacity, or prefix structures that increase serving cost. These issues matter because SIDs are not merely features. They define the address space exposed to the generator.

This gap is especially awkward for resource reuse. A new tokenizer repository may expose checkpoints, item mappings, codebooks, or only intermediate feature files. Downstream users then need to answer basic engineering questions before training a generator: do all catalog items have valid codes, are many items aliased to the same full code, are collisions concentrated in the tail, and do prefix neighborhoods reflect user co-occurrence rather than only metadata taxonomy? Today these checks are usually reimplemented ad hoc inside each paper. The resulting evidence is difficult to compare because each method reports a different mixture of codebook usage, downstream ranking, and qualitative examples.

We present AUDIT-SID, a resource for auditing item-to-SID tokenizer artifacts before or alongside downstream recommendation evaluation. The goal is deliberately narrower than proposing a new tokenizer. AUDIT-SID asks a concrete question: given a public tokenizer artifact that maps items to code sequences, what can be said about its capacity use, collisions, behavioral alignment, head-tail allocation, and deployment cost without retraining a full generator?

The resource contribution is threefold. First, AUDIT-SID defines a small mapping-first interface for SID assignments, item metadata, interaction histories, and optional generator outputs. Second, it implements D1–D5a diagnostics for utilization, collisions, semantic-collaborative alignment, head-tail capacity, and structural serving-cost proxies, with D6 churn support for continual-tokenizer settings. Third, it demonstrates the audit on public GRID/RQ-KMeans-style, ReSID/GAOQ, sanity, and controlled-stressor artifacts. The same-item Musical case study and stressor table support diagnostic claims: they separate full-code collision pressure, interaction-qualified collision risk, collaborative-prefix recovery, tail capacity, and structural prefix fan-out before a downstream generator is trained. The paper follows the CIKM Resource four-page constraint, where appendices

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

count toward the limit and detailed logs are therefore placed in the online artifact [2].

2 Toolkit and Diagnostics

AUDIT-SID treats a tokenizer export as an artifact, not as an opaque component of a trained recommender. The minimum input is a table `sid_assignments(item_id, sid_0, ..., sid_L)`. Optional tables provide item metadata, interaction histories, and generator outputs. This contract lets adapters normalize heterogeneous repositories while keeping the audit independent of a particular training loop. Figure 1 shows how heterogeneous exports become comparable reviewer artifacts without retraining a downstream generator.

Artifact contract. The contract has three required checks and two optional extensions. The required checks are: every row must contain a stable item key, every SID level must be typed as a discrete token, and the item universe used by diagnostics must be joinable to the metadata and interaction slices. The optional metadata table supports semantic or category purity diagnostics. The optional interaction table supports collaborative-neighborhood diagnostics and head-tail splits. The optional `generator_outputs` table is deliberately separated from the mapping contract because many public tokenizer releases expose item codes without trained generator traces.

Validation before metrics. AUDIT-SID refuses to treat an adapter as evidence until it passes coverage and provenance checks: item-count agreement, missing-code counts, duplicate item keys, depth consistency, and declared caveats for feature sources or bounded training runs. This matters for the present resource paper because controlled exports, sanity tokenizers, and named-method mappings can all be loaded by the same metric runner but support different claims. The manifest therefore records whether a row is a runnable named method, a controlled stressor, an optional extension, or literature-only coverage context.

D1: utilization. AUDIT-SID reports per-level usage, prefix counts, entropy-style imbalance summaries, and dead or underused code indicators. These expose codebook collapse and uneven capacity even when full ranking metrics are unavailable.

D2: collision profile. AUDIT-SID measures duplicate full SIDs and prefix collisions at each depth. The reported full-collision rate is the fraction of items whose complete SID is shared by at least one other item. In the current resource version, D2 is a collision profile rather than a causal harm estimate. To make this boundary visible, the artifact also includes a bounded interaction-qualified controller inspired by qualification-aware collision work [5]. The main metric still reports collisions because full code aliasing is an artifact-level fact: two items assigned the same complete address cannot be distinguished by a generator that emits only that address.

D3: semantic-collaborative alignment. Semantic grouping and collaborative usefulness need not agree. AUDIT-SID therefore computes a co-occurrence-based collaborative neighborhood proxy from training interactions and asks how often prefix neighborhoods recover each item's top-20 co-occurring neighbors. Metadata purity

is retained only as an auxiliary diagnostic, not as the definition of collaborative quality.

D4: head-tail capacity. AUDIT-SID splits items into popularity buckets and measures whether full SID capacity and prefix structure are allocated differently across head, mid, and tail items. This helps separate genuine tokenizer behavior from simply restating the catalog popularity distribution.

D5a, D6, and D7. D5a reports structural cost proxies: SID length, level-wise prefix fan-out, duplicate codes, and trie-like expansion pressure. These are not real generator serving latencies. D6 computes churn between tokenizer refreshes and is useful for drift-aware continual-tokenization settings [3]. D7 is an interface hook for generator behavior, such as invalid paths, next-token entropy, and duplicate generated candidates; it requires `generator_outputs` and is not claimed by the current evidence.

Identifier foundations and recommender diagnostic frameworks motivate the resource framing, but they are not runnable tokenizer rows. Table 1 therefore focuses on SID artifact facets and marks which parts are demonstrated in v0.

3 Resource Demo and Case Study

The demo evaluates the artifact interface rather than ranking tokenizer systems. It combines two public SID export paths with controlled rows that calibrate the diagnostics. GRID supplies the open RQ-KMeans-style path [7]; ReSID supplies the GAOQ line and processed Musical artifacts [9]. Sanity tokenizers and stressors are included only when they clarify what a diagnostic can and cannot identify.

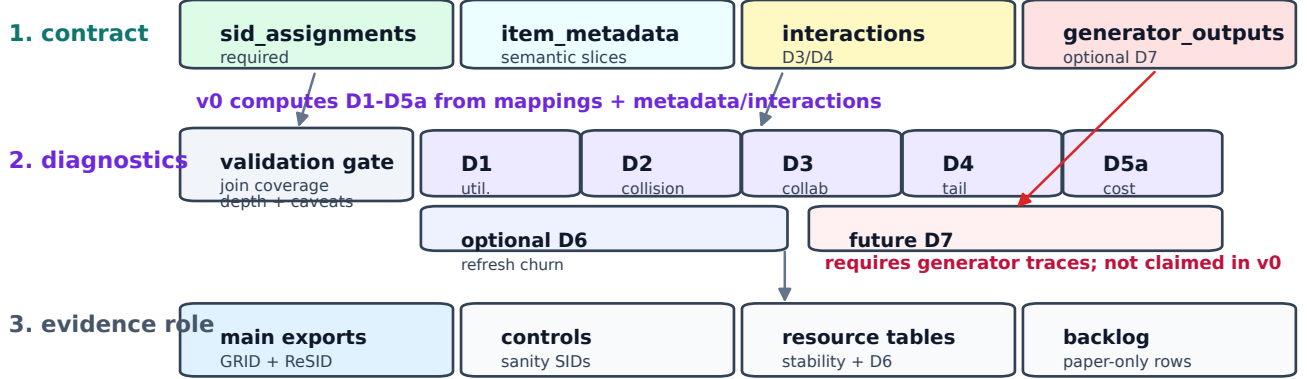
Case-study protocol. The paper-facing case study uses the same 23,742 Musical items for both rows. The GRID row is a controlled feature-text export: ReSID processed feature text is embedded and passed through the official GRID-style residual k-means module. The ReSID row is a bounded FAMA-to-GAOQ export on the same item universe. This protocol is intentionally modest. It gives reviewers a same-item diagnostic contrast while avoiding the stronger, unsupported claim that the paper reproduces every training choice of TIGER, GRID, or the full ReSID pipeline.

Table 2: Same-item Musical artifact profiles on 23,742 items. Higher unique full codes, D3 L1 co-occurrence prefix recall, and D4 tail unique-SID ratio are better; lower D2 full-collision rate is better. D5a prefixes are level-wise prefix counts, not latency. This is not a downstream leaderboard: GRID feature-text uses ReSID processed feature text, and ReSID is a bounded GAOQ export.

System	Unique	D2 full	D3 L1	D4 tail	D5a prefixes
GRID feature-text	3,749	0.9769	0.0552	0.3695	64/3440/3749
ReSID bounded	23,742	0.0000	0.1535	1.0000	32/1280/23742

Table 2 compares two exports on the same 23,742 Musical items. The GRID row is intentionally labeled as feature-text: it uses processed feature text from the ReSID artifact path and the official

AUDIT-SID maps heterogeneous SID exports to comparable audit evidence



Claim boundary: mapping/diagnostic evidence now; downstream generator quality later.

Figure 1: AUDIT-SID turns heterogeneous SID tokenizer exports into comparable audit artifacts. Exports are normalized into a mapping-first contract, validated for joinability and provenance, and summarized through D1–D5a diagnostics. D6 is optional churn evidence; D7 is enabled only when generator outputs or beam traces are provided.

Table 1: Facet coverage matrix. ‘M’ denotes main PDF evidence, ‘A’ artifact repository evidence, ‘H’ interface hook, and ‘–’ no v0 evidence. The table is coverage context, not a claim that all anchor methods are reproduced.

Facet	Anchors	v0 role	D1	D2	D3	D4	D5a	D6	D7
A. Canonical residual SID	TIGER, GRID, RQ-KMeans	GRID feature-text	M	M	M	M	M	–	–
B1. Collaborative / predictability	ReSID, CoST, LETTER, DiscRec	ReSID bounded	M	M	M	M	M	–	–
B2. Collision / capacity	QuaSID, AdaSID, CARD, CapsID	stressors/backlog	A	A	–	A	A	–	–
B3. Ranking / retrieval	DIGER, joint search-rec SID	interface hook	–	–	H	–	H	–	H
B4. Bottleneck / interface	AsymRec, CapsID, Long SID	future target	–	–	–	H	H	–	H
C. Drift / staleness	DACT, SID staleness	optional smoke	–	–	–	–	–	A	–
D. Deployment / search	Snapchat SID, unified search-rec SID	motivation	–	–	–	–	H	–	H
R0/control layer	RecList, sanity SIDs, MovieLens smoke	controls	A	A	A	A	A	–	–

MiniBatchKMeans-style residual quantization module, not a faithful raw-text TIGER reproduction. Under that controlled setup, the row exposes high collision pressure and limited tail capacity. The ReSID/GAOQ row exports one full code per item in this bounded run, yielding zero full-code collisions and higher D3-L1 collaborative prefix recovery. The D5a prefix column reports level-wise prefix counts, a structural proxy for trie expansion rather than real serving latency. The same GRID Musical feature-text export was rerun locally with seeds 43 and 44; across seeds 42–44, duplicate-SID rate remains 0.8327–0.8421 and full-collision rate remains 0.9751–0.9769, so the collision pressure in Table 2 is not a single-seed artifact.

Table 3: Controlled stressor signals. These rows are not named-tokenizer evidence; they test whether D2, D4, and D5a react to specific artifact mechanisms.

Stressor	Metric	Signal 1	Signal 2
Qualified collision	co-occurrence lift	3.86x	1.19x
Capacity budget	unique ratio	1.000 head	0.028 tail
Variable depth	prefix-cost view	active	max-depth

Diagnostic findings. Tables 2 and 3 support three artifact-level findings. First, collision pressure is stable in the controlled GRID

row: most items share a complete SID, whereas the same interface ingests a collision-free ReSID mapping. The qualified-collision stressor then shows why raw collision volume is insufficient: collisions that overlap user co-occurrence have a 3.86x lift, while synthetic hash collisions have 1.19x. Second, collision-free addresses and collaborative-prefix alignment are different objectives. ReSID has no full collisions and higher D3-L1 than the GRID row, while a hand-designed category-prefix control in the repository recovers more level-1 co-occurrence neighbors without being a learned recommender tokenizer. Third, D4 and D5a separate capacity allocation from prefix structure: in the capacity stressor, head uniqueness reaches 1.000 while tail uniqueness remains 0.028, and variable-depth stressors show that active prefix cost need not match a fixed maximum-depth view.

Artifact-repository evidence. The remaining repository tables extend the same checks without changing the paper claims. Sanity rows calibrate D1–D5a, GRID scale rows test larger item counts, DACT rows exercise optional D6 churn, and MovieLens rows check non-Amazon schema portability. These tables support artifact usability; Table 2 remains the only method-facing case study in the main paper.

Table 4: Reviewer-facing artifact checklist. Columns indicate whether each asset is inspectable in a clean checkout, executable without private caches, and tied to a PDF claim.

Asset group	Inspect	Run	PDF claim
Quickstart/license	yes	–	package
Unit tests/verifier	yes	yes	code path
Figure/table CSVs	yes	yes	Tables 1–3
Manifest/claim audit	yes	–	row roles
Extra CSV panels	yes	–	support

4 Availability and Limitations

The artifact repository contains the AUDIT-SID Python package, adapter scripts, metric runners, generated CSV, Markdown, and $\LaTeX$ tables, provenance logs, the BibTeX/source audit used for this draft, an MIT license, and a reviewer quickstart. The reviewer entry point is the tag `audit-sid-cikm-resource-v0.1`, which exposes a clean-checkout verifier for the published tables and figure. Large local artifacts are kept out of git and described through manifests so that reviewers can distinguish public reproducibility assets from local caches.

The evidence supports a conservative resource-demo claim. The toolkit does not reproduce TIGER end-to-end; the GRID Musical row is a controlled feature-text export. The ReSID Sports balanced GAOQ path was stopped after constrained k-means became CPU-bound, so the main ReSID evidence is the bounded Musical export. CARD remains a fallback/stressor path, not faithful CARD reproduction [14]. D2 includes a bounded interaction-qualified controller but not causal harm; D3 is not proven monotonic with Recall@K or NDCG; D5a is structural rather than measured latency; and D6 is optional drift evidence.

These boundaries define the resource’s scope. AUDIT-SID audits tokenizer artifacts; it is not a new SID tokenizer, a complete generative-recommender evaluation, or an industrial online-impact study. Artifact diagnostics can reveal pressure points before generator training, but they do not replace ranking evaluation. Future versions should add causal collision-harm validation, generator-output diagnostics, stronger same-dataset method coverage, and unified search/recommendation checks [10]. Industrial SID deployment studies further motivate treating these artifact properties as engineering objects [8].

Reviewer workflow and claim discipline. Reviewers inspect the manifest to distinguish named-method, controlled, optional, and literature-only rows, run the clean-checkout verifier, and compare published CSVs with the paper claims. Full metric rebuilds are optional and require local caches. The resource separates *mapping statements* directly supported by SID-assignment tables, *diagnostic statements* that join mappings with metadata or interactions, and *system-quality statements* that require generator training, ranking evaluation, or serving traces. This paper makes the first two kinds of statements and explicitly not the third.

Maintenance plan. New adapters emit the same normalized tables, declare their evidence role, and record feature or checkpoint provenance. Versioned manifests and fixed-name “latest” reports

preserve stable entry points and timestamped provenance. The Musical case study is therefore not a ranking of GRID against ReSID; it shows how one artifact contract turns collisions, collaborative prefix recovery, tail allocation, and prefix fan-out into separately checkable review objects.

References

- [1] Patrick John Chia, Maurizio Ferrari Dacrema, Maria Soledad Pera, and Jacopo Tagliafue. 2022. Beyond NDCG: Behavioral Testing of Recommender Systems with RecList. In *Companion Proceedings of the ACM Web Conference 2022*. arXiv:2111.09963 [cs.IR] doi:10.1145/3487553.3524215 arXiv:2111.09963.
- [2] CIKM 2026 Organizing Committee. 2026. CIKM 2026 Resource Papers. <https://cikm2026.diag.uniroma1.it/resource-papers/>. Accessed 2026-05-19.
- [3] Yuebo Feng, Jiahao Liu, Mingzhe Han, Dongsheng Li, Hansu Gu, Peng Zhang, Tun Lu, and Ning Gu. 2026. Drift-Aware Continual Tokenization for Generative Recommendation. *arXiv preprint arXiv:2603.29705* (2026). arXiv:2603.29705 [cs.IR] doi:10.48550/arXiv.2603.29705
- [4] Junchen Fu, Xuri Ge, Alexandros Karatzoglou, Ioannis Arapakis, Suzan Verberne, Joemon M. Jose, and Zhaochun Ren. 2026. Differentiable Semantic ID for Generative Recommendation. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*. arXiv:2601.19711 [cs.IR] doi:10.48550/arXiv.2601.19711 Accepted by SIGIR 2026; arXiv:2601.19711.
- [5] Zheng Hu, Yuxin Chen, Yongsan Pan, Xu Yuan, Yuting Yin, Daoyuan Wang, Boyang Xia, Zefei Luo, Hongyang Wang, Songhao Ni, Dongxu Liang, Jun Wang, Shimin Cai, Tao Zhou, Fuji Ren, and Wenwu Ou. 2026. Stop Treating Collisions Equally: Qualification-Aware Semantic ID Learning for Recommendation at Industrial Scale. *arXiv preprint arXiv:2603.00632* (2026). arXiv:2603.00632 [cs.IR] doi:10.48550/arXiv.2603.00632
- [6] Wenyue Hua, Yingqiang Ge, Shuyuan Xu, Zelong Ji, Jiayue Chen, and Yongfeng Zhang. 2023. How to Index Item IDs for Recommendation Foundation Models. *arXiv preprint arXiv:2305.06569* (2023). arXiv:2305.06569 [cs.IR] doi:10.48550/arXiv.2305.06569
- [7] Clark Mingxuan Ju, Liam Collins, Leonardo Neves, Bhuvish Kumar, Louis Yufeng Wang, Tong Zhao, and Neil Shah. 2025. Generative Recommendation with Semantic IDs: A Practitioner's Handbook. *arXiv preprint arXiv:2507.22224* (2025). arXiv:2507.22224 [cs.IR] doi:10.48550/arXiv.2507.22224
- [8] Clark Mingxuan Ju, Andy Lin, Adedamola Oyewole, Zeel Patel, Hyunjin Park, Jerome Arje, Zhe Dong, Anuj Bhardwaj, Rahul Rastogi, and Neil Shah. 2026. Semantic IDs for Recommendation Systems. *arXiv preprint arXiv:2604.03949* (2026). arXiv:2604.03949 [cs.IR] doi:10.48550/arXiv.2604.03949
- [9] Yu Liang, Zhongjin Zhang, Yuxuan Zhu, Kerui Zhang, Zhiluohan Guo, Wenhong Zhou, Zonqi Yang, Kangle Wu, Yabo Ni, Anxiang Zeng, Cong Fu, Jianxin Wang, and Jiazhi Xia. 2026. Rethinking Generative Recommender Tokenizer: Recsys-Native Encoding and Semantic Quantization Beyond LLMs. *arXiv preprint arXiv:2602.02338* (2026). arXiv:2602.02338 [cs.IR] doi:10.48550/arXiv.2602.02338
- [10] Gustavo Penha, Edoardo D'Amico, Marco De Nadai, Enrico Palumbo, Alexandre Tamborrino, Ali Vardasbi, Max Lefarov, Shawn Lin, Timothy Heath, Francesco Fabbri, and Hugues Bouchard. 2025. Semantic IDs for Joint Generative Search and Recommendation. In *Proceedings of the 19th ACM Conference on Recommender Systems*. arXiv:2508.10478 [cs.IR] doi:10.48550/arXiv.2508.10478 Late-Breaking Results track; arXiv:2508.10478.
- [11] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H. Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. 2023. Recommender Systems with Generative Retrieval. In *Advances in Neural Information Processing Systems*. arXiv:2305.05065 [cs.IR] doi:10.48550/arXiv.2305.05065
- [12] Adit Krishan Singh, Vamsi Meduri, Pranjal Awasthi, Yuanyuan Feng, Amee Trivedi, Afshin Rostamizadeh, and Sanjiv Kumar. 2023. Better Generalization with Semantic IDs: A Case Study in Ranking for Recommendations. *arXiv preprint arXiv:2306.08121* (2023). arXiv:2306.08121 [cs.IR] doi:10.48550/arXiv.2306.08121
- [13] Bowen Wang, Shizhe Diao, Yan Feng, Baijun He, Chen Li, Xiyuan Wang, Jingang Fu, Haiyang Lyu, and Yonggang Chen. 2024. LETTER: A LLM-Enhanced Two-Tower Recommender System. *arXiv preprint arXiv:2405.07314* (2024). arXiv:2405.07314 [cs.IR] doi:10.48550/arXiv.2405.07314
- [14] Yibiao Wei, Jie Zou, Pengfei Zhang, Xiao Ao, Weikang Guo, Zeyu Ma, and Yang Yang. 2026. CARD: Non-Uniform Quantization of Visual Semantic Unit for Generative Recommendation. *arXiv preprint arXiv:2604.26427* (2026). arXiv:2604.26427 [cs.IR] doi:10.48550/arXiv.2604.26427
- [15] Han Zhu, Hao Liu, Zhiwei Liu, Wenqi Sun, Yingxue Zhang, Xinyang Fu, Weijie Liu, and Hui Xiong. 2024. Collaborative Semantic Tokenization for Generative Retrieval. *arXiv preprint arXiv:2404.14774* (2024). arXiv:2404.14774 [cs.IR] doi:10.48550/arXiv.2404.14774

GenAI Usage Disclosure

The authors used generative AI tools for drafting assistance, code-review support, and experiment-log organization. The reported claims, tables, and citations were checked against the artifact repository and cited sources during paper preparation.